```python
import pandas as pd
import warnings
import altair as alt
warnings.filterwarnings('ignore')
alt.renderers.enable("png")
alt.data_transformers.disable_max_rows()

Crime = pd.read_csv('data/Crimes.csv')

Crime.head()

pd.set_option("display.max_columns", None)
pd.set_option("display.max_colwidth", None)
pd.set_option("display.width", None)

Crime.head(5)
```

	ID	Case Number	Date	Block	IUCR	Primary Type
0	14075483	JK105557	12/31/2025 11:58:00 PM	050XX S PAULINA ST	0560	ASSAULT
1	14070845	JK100006	12/31/2025 11:54:00 PM	013XX W LAKE ST	0454	BATTERY
2	14071367	JK100694	12/31/2025 11:50:00 PM	005XX S MICHIGAN AVE	0460	BATTERY
3	14070841	JK100162	12/31/2025 11:50:00 PM	068XX S PERRY AVE	0486	BATTERY
4	14070846	JK100015	12/31/2025 11:47:00 PM	004XX W SURF ST	0486	BATTERY

```python
import geopandas as gpd
from shapely.geometry import Point

tracts = gpd.read_file("/Users/harryfranklin/Downloads/Census_20Tracts/Census_Tracts.shp")

geometry = [
    Point(xy) for xy in zip(Crime["Longitude"], Crime["Latitude"])
]

crimes_gdf = gpd.GeoDataFrame(
    Crime,
    geometry=geometry,
    crs="EPSG:4326"
)

tracts = tracts.to_crs(crimes_gdf.crs)
```

```
crimes_with_tracts = gpd.sjoin(
    crimes_gdf,
    tracts,
    how="left",
    predicate="within"
)

crimes_with_tracts
```

	ID	Case Number	Date	Block	IUCR	Primary
0	14075483	JK105557	12/31/2025 11:58:00 PM	050XX S PAULINA ST	0560	ASSAU
1	14070845	JK100006	12/31/2025 11:54:00 PM	013XX W LAKE ST	0454	BATTE
2	14071367	JK100694	12/31/2025 11:50:00 PM	005XX S MICHIGAN AVE	0460	BATTE
3	14070841	JK100162	12/31/2025 11:50:00 PM	068XX S PERRY AVE	0486	BATTE
4	14070846	JK100015	12/31/2025 11:47:00 PM	004XX W SURF ST	0486	BATTE
...	...	...	...	...	...	...
334521	12258861	JE100369	01/01/2021 12:00:00 AM	016XX N CICERO AVE	051A	ASSAU
334522	12259383	JE100487	01/01/2021 12:00:00 AM	112XX S HALSTED ST	0486	BATTE
334523	12307445	JE159529	01/01/2021 12:00:00 AM	035XX W MONROE ST	0265	CRIMIN
334524	12258747	JE100294	01/01/2021 12:00:00 AM	123XX S HALSTED ST	0486	BATTE
334525	12319911	JE174516	01/01/2021 12:00:00 AM	024XX W 66TH ST	0266	CRIMIN

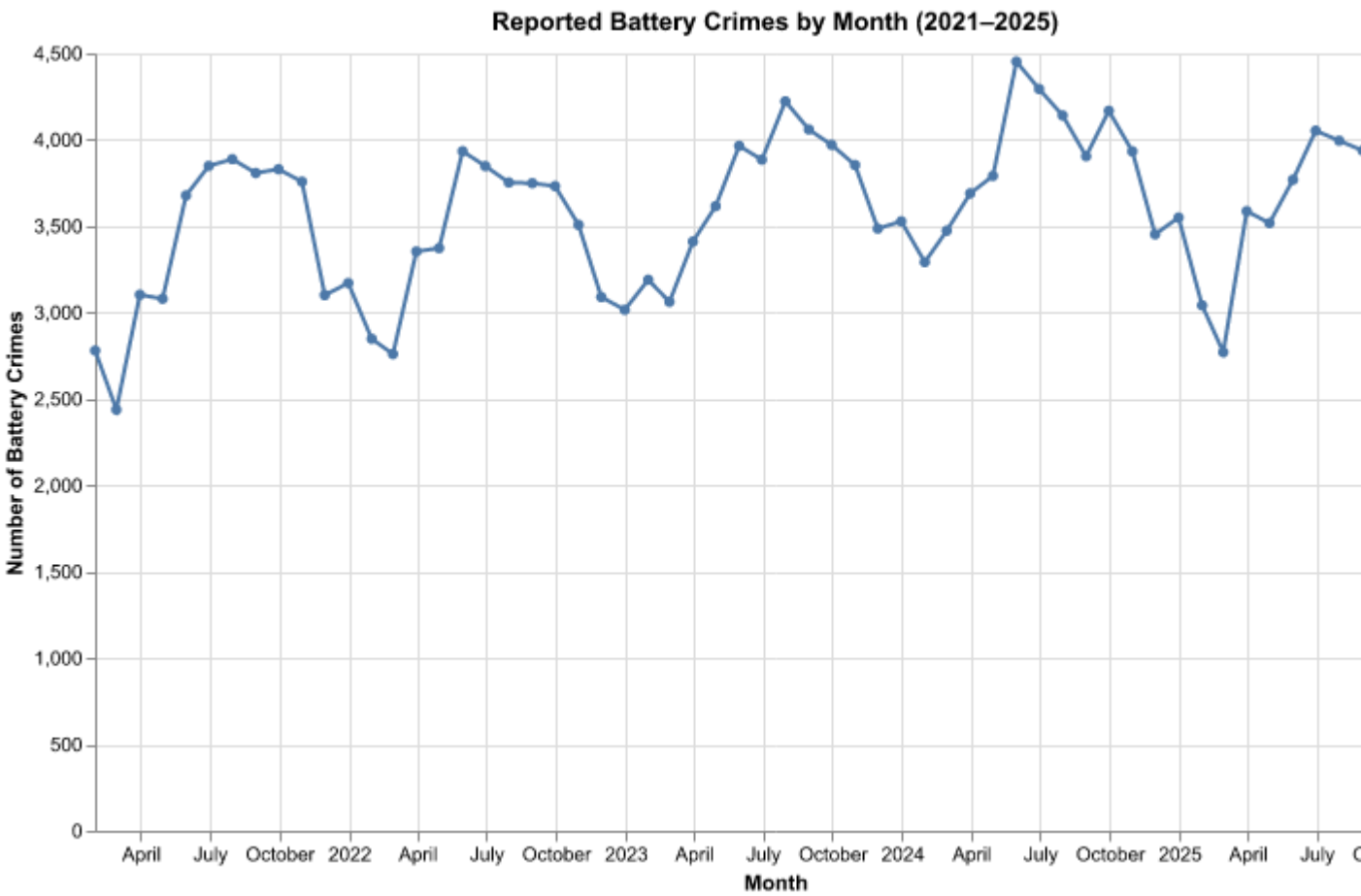
```
Crime = crimes_with_tracts.iloc[:, :-14]
Crime["Date"] = pd.to_datetime(Crime["Date"])

Battery = Crime[Crime['Primary Type'] == 'BATTERY']
Assault = Crime[Crime['Primary Type'] == 'ASSAULT']
SA = Crime[Crime['Primary Type'].isin(["CRIMINAL SEXUAL ASSAULT", "CRIM SEXUAL ASSAULT"])]
Homicide = Crime[Crime['Primary Type'] == 'HOMICIDE']
DV = Crime[Crime['Primary Type'] == 'DOMESTIC VIOLENCE']

Pop2023 = pd.read_csv("/Users/harryfranklin/Downloads/Population.csv")
Crime["CENSUS_T_1"] = Crime["CENSUS_T_1"].astype(str)
Pop2023["GEOID"] = Pop2023["GEOID"].astype(str)
Crime = Crime.merge(
    Pop2023[["GEOID", "Population"]],
    left_on="CENSUS_T_1",
    right_on="GEOID",
    how="left"
)
```

```
Crime = Crime.drop(
    columns=[col for col in Crime.columns if "GEOID" in col],
    errors="ignore"
)

Crime
```

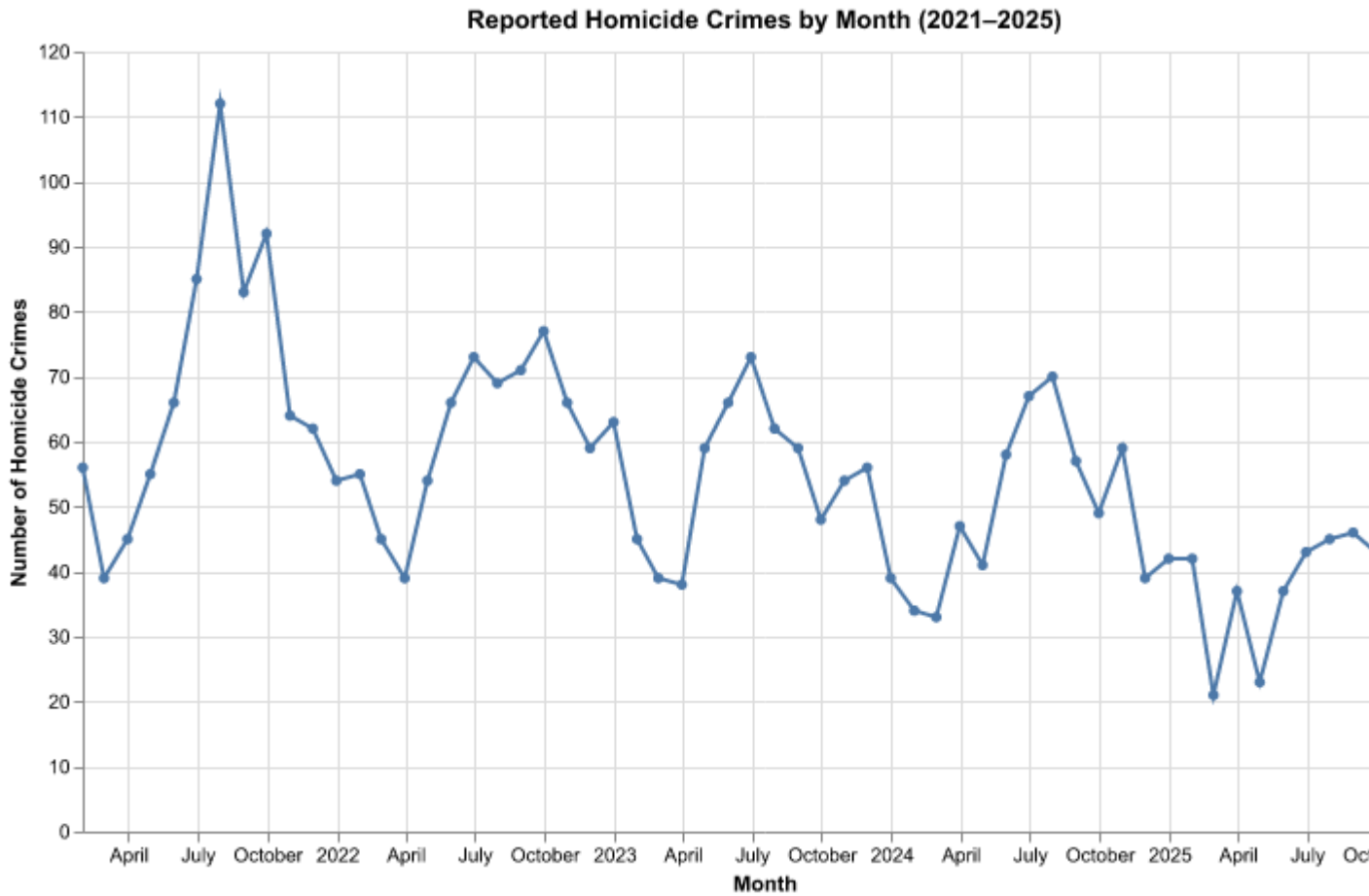
	ID	Case Number	Date	Block	IUCR	Primary Typ
0	14075483	JK105557	2025-12-31 23:58:00	050XX S PAULINA ST	0560	ASSAULT
1	14070845	JK100006	2025-12-31 23:54:00	013XX W LAKE ST	0454	BATTERY
2	14071367	JK100694	2025-12-31 23:50:00	005XX S MICHIGAN AVE	0460	BATTERY
3	14070841	JK100162	2025-12-31 23:50:00	068XX S PERRY AVE	0486	BATTERY
4	14070846	JK100015	2025-12-31 23:47:00	004XX W SURF ST	0486	BATTERY
...	...	...	...	...	...	...
334521	12258861	JE100369	2021-01-01 00:00:00	016XX N CICERO AVE	051A	ASSAULT
334522	12259383	JE100487	2021-01-01 00:00:00	112XX S HALSTED ST	0486	BATTERY
334523	12307445	JE159529	2021-01-01 00:00:00	035XX W MONROE ST	0265	CRIMINAL S
334524	12258747	JE100294	2021-01-01 00:00:00	123XX S HALSTED ST	0486	BATTERY
334525	12319911	JE174516	2021-01-01 00:00:00	024XX W 66TH ST	0266	CRIMINAL S

```
alt.data_transformers.disable_max_rows()
monthly = (
    Battery
    .groupby(pd.Grouper(key="Date", freq="M"))
    .size()
    .reset_index(name="monthly_count")
)

chart1 = (
    alt.Chart(monthly)
    .mark_line(point=True)
    .encode(
        x=alt.X(
            "Date:T",
            title="Month"
        ),
        y=alt.Y(
            "monthly_count:Q",
            title="Number of Battery Crimes"
        ),
```

```
        tooltip=[
            alt.Tooltip("Date:T", title="Month"),
            alt.Tooltip("monthly_count:Q", title="Count")
        ]
    )
    .properties(
        title="Reported Battery Crimes by Month (2021-2025)",
        width=700,
        height=400
    )
)

chart1
```

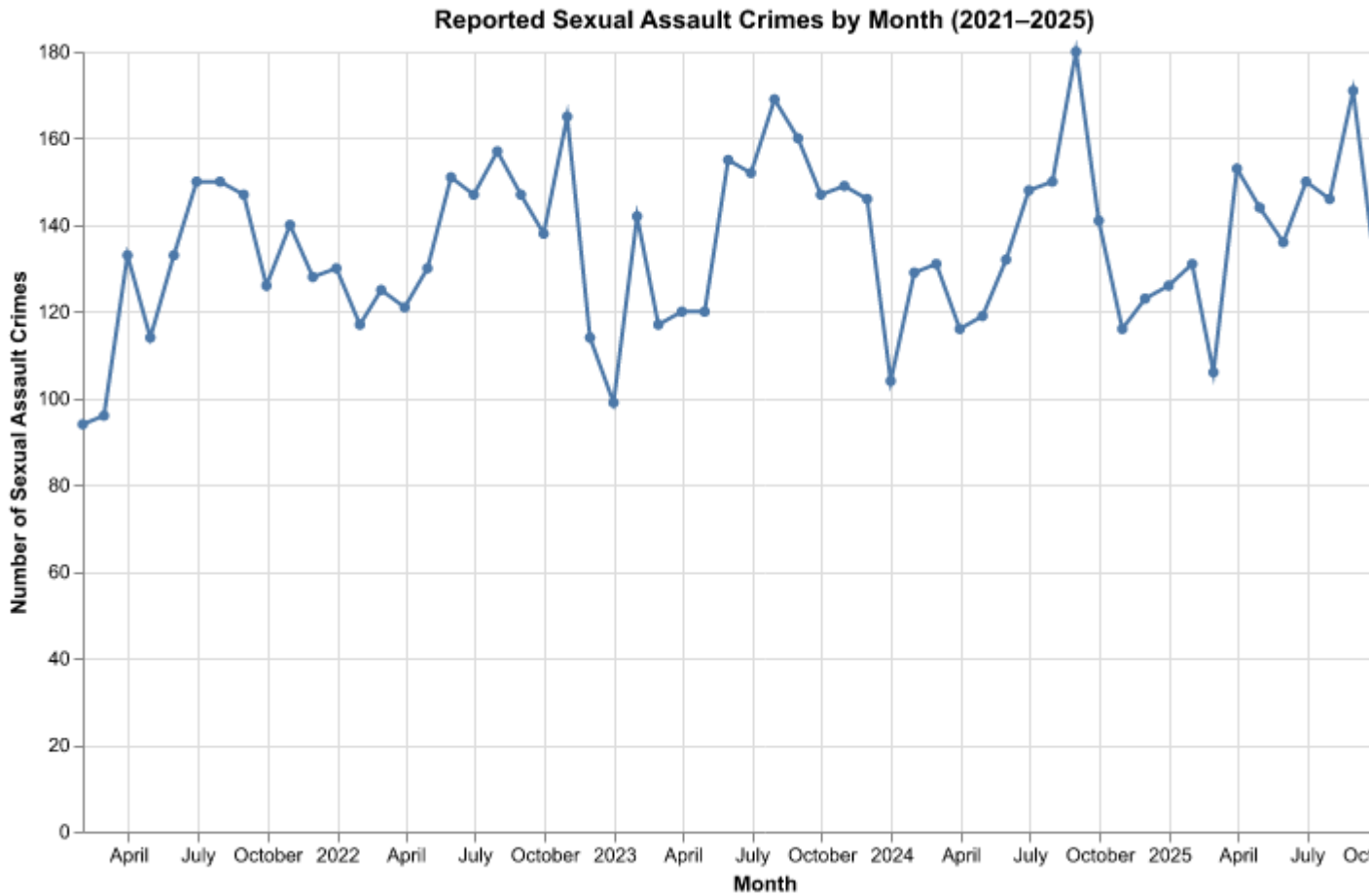


```python
monthly = (
    Homicide
    .groupby(pd.Grouper(key="Date", freq="M"))
    .size()
    .reset_index(name="monthly_count")
)

chart2 = (
    alt.Chart(monthly)
    .mark_line(point=True)
    .encode(
        x=alt.X(
            "Date:T",
            title="Month"
        ),
        y=alt.Y(
            "monthly_count:Q",
            title="Number of Homicide Crimes"
        ),
        tooltip=[
            alt.Tooltip("Date:T", title="Month"),
            alt.Tooltip("monthly_count:Q", title="Count")
        ]
    )
    .properties(
        title="Reported Homicide Crimes by Month (2021-2025)",
        width=700,
        height=400
    )
)

chart2
```



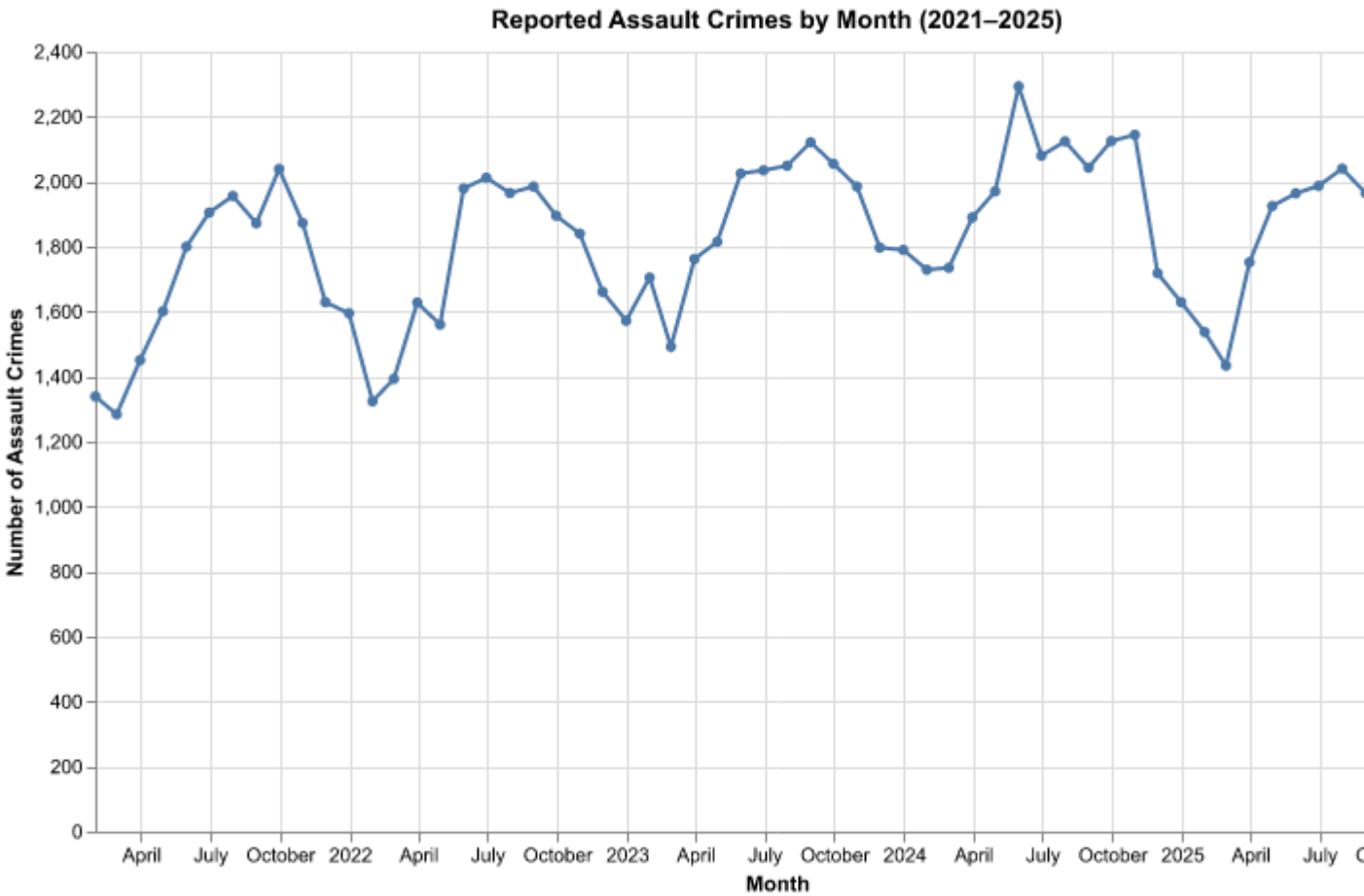

```
monthly = (
    SA
    .groupby(pd.Grouper(key="Date", freq="M"))
    .size()
    .reset_index(name="monthly_count")
)

chart3 = (
    alt.Chart(monthly)
    .mark_line(point=True)
    .encode(
        x=alt.X(
            "Date:T",
            title="Month"
        ),
        y=alt.Y(
```

```
            "monthly_count:Q",
            title="Number of Sexual Assault Crimes"
        ),
        tooltip=[
            alt.Tooltip("Date:T", title="Month"),
            alt.Tooltip("monthly_count:Q", title="Count")
        ]
    )
    .properties(
        title="Reported Sexual Assault Crimes by Month (2021-2025)",
        width=700,
        height=400
    )
)

chart3
```



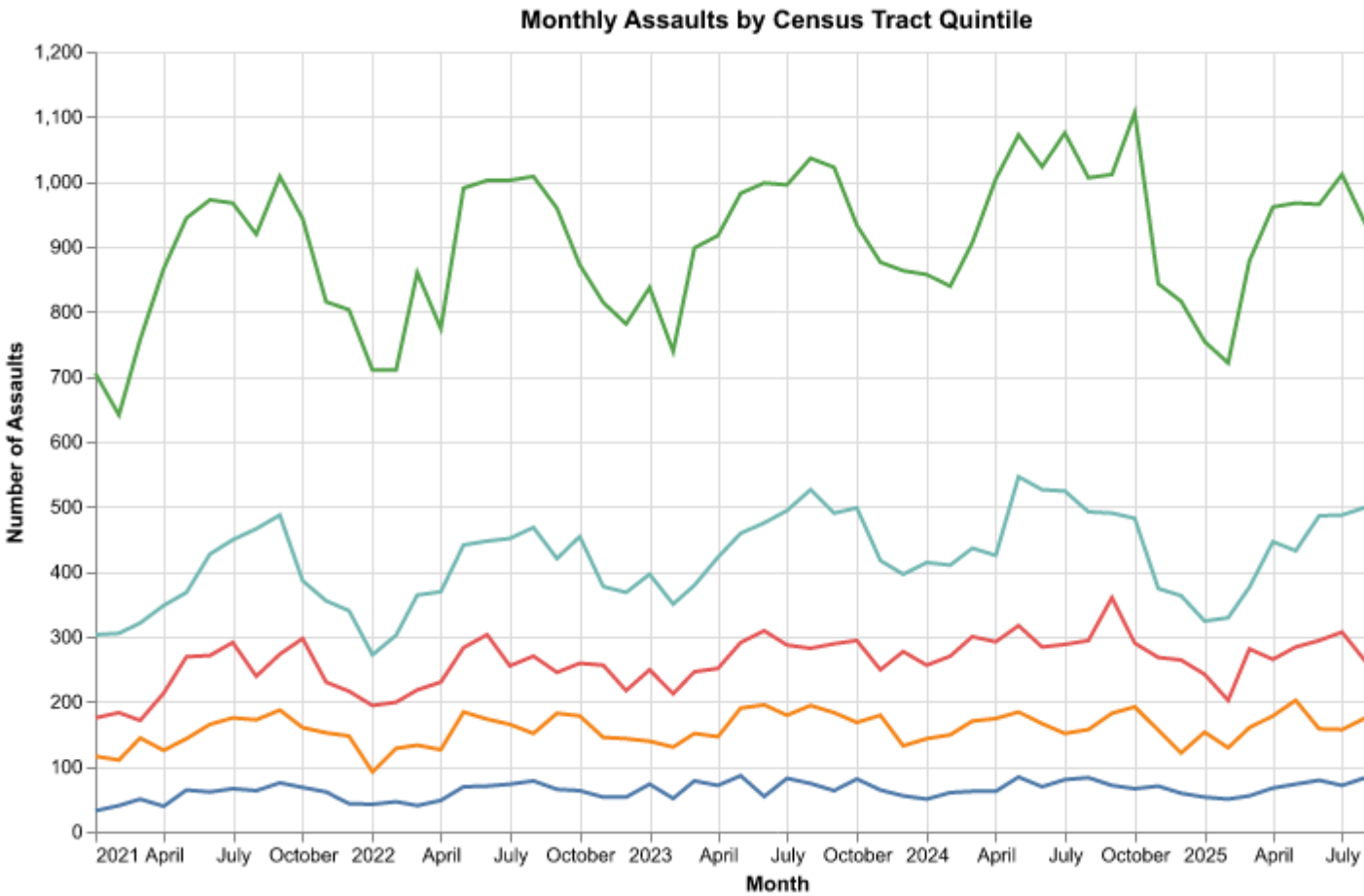

```
monthly = (
    Assault
    .groupby(pd.Grouper(key="Date", freq="M"))
    .size()
    .reset_index(name="monthly_count")
)

chart4 = (
    alt.Chart(monthly)
    .mark_line(point=True)
    .encode(
        x=alt.X(
            "Date:T",
            title="Month"
        ),
        y=alt.Y(
```

```
            "monthly_count:Q",
            title="Number of Assault Crimes"
        ),
        tooltip=[
            alt.Tooltip("Date:T", title="Month"),
            alt.Tooltip("monthly_count:Q", title="Count")
        ]
    )
    .properties(
        title="Reported Assault Crimes by Month (2021-2025)",
        width=700,
        height=400
    )
)

chart4
```



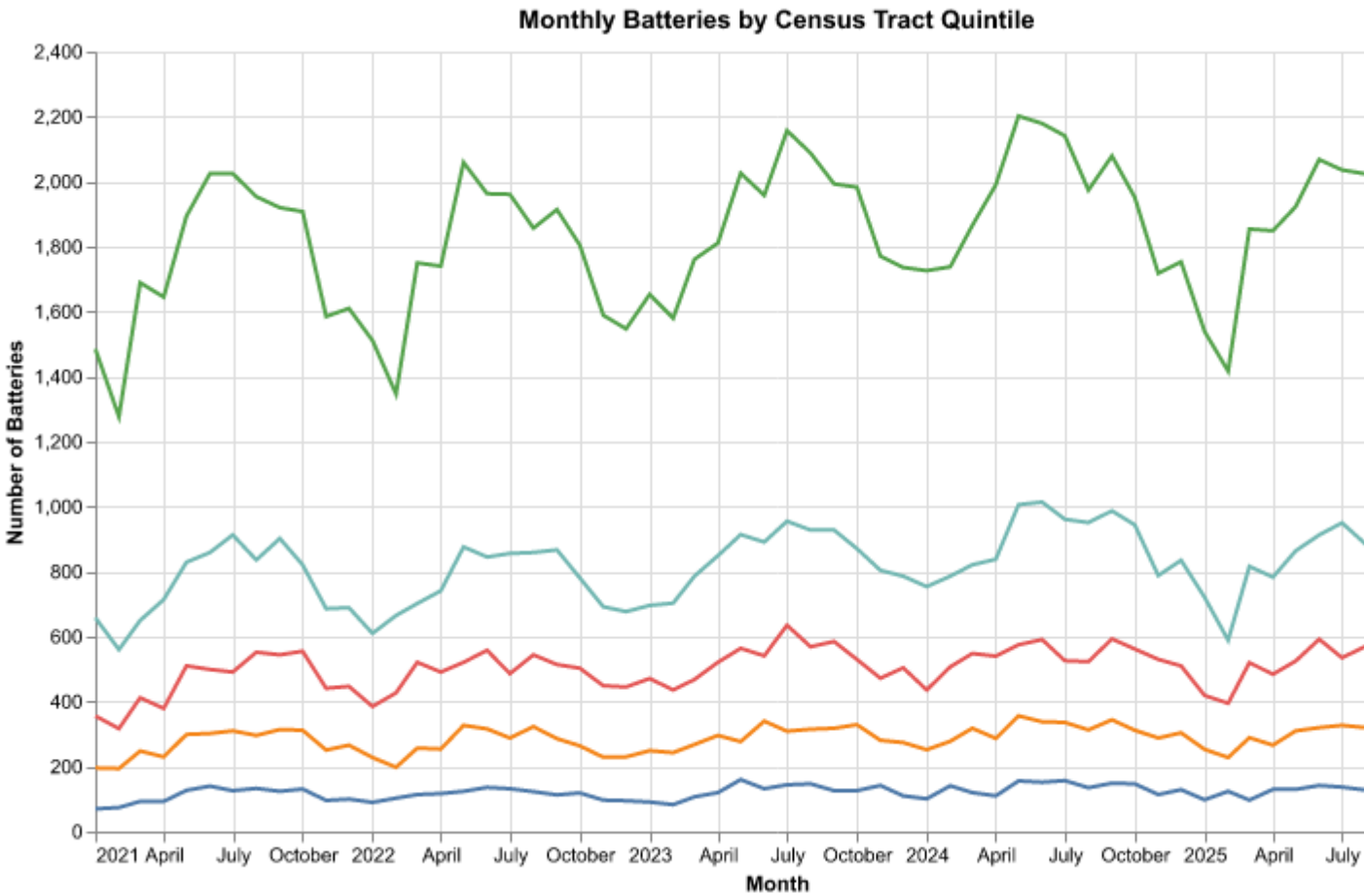

```
Assault["Date"] = pd.to_datetime(Assault["Date"])

if "quintile" in Assault.columns:
    Assault = Assault.drop(columns=["quintile"])

tract_counts = (
    Assault
    .groupby("CENSUS_T_1")
    .size()
    .reset_index(name="total_assaults")
)

tract_counts["quintile"] = pd.qcut(
    tract_counts["total_assaults"],
    5,
    labels=["Q1 (Lowest)", "Q2", "Q3", "Q4", "Q5 (Highest)"]
```

```
)

Assault = Assault.merge(
    tract_counts[["CENSUS_T_1", "quintile"]],
    on="CENSUS_T_1",
    how="left"
)

Assault["month"] = Assault["Date"].dt.to_period("M").dt.to_timestamp()

quintile_time = (
    Assault
    .groupby(["month", "quintile"])
    .size()
    .reset_index(name="assault_count")
)

chart5 = (
    alt.Chart(quintile_time)
    .mark_line()
    .encode(
        x=alt.X("month:T", title="Month"),
        y=alt.Y("assault_count:Q", title="Number of Assaults"),
        color=alt.Color("quintile:N", title="Census Tract Quintile"),
    )
    .properties(
        width=700,
        height=400,
        title="Monthly Assaults by Census Tract Quintile"
    )
)

chart5
```



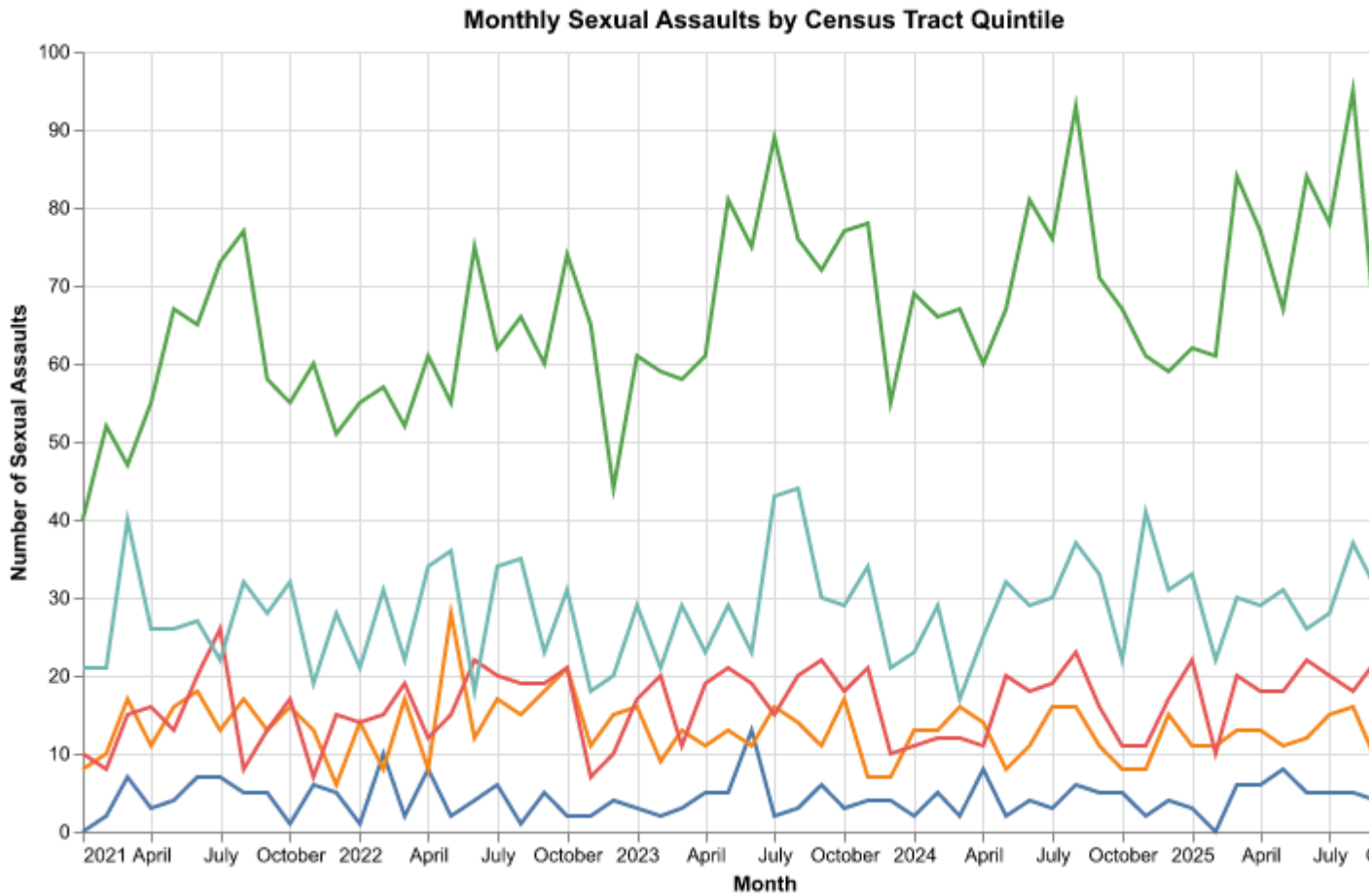

```
Battery["Date"] = pd.to_datetime(Battery["Date"])

if "quintile" in Battery.columns:
    Battery = Battery.drop(columns=["quintile"])

tract_counts = (
    Battery
    .groupby("CENSUS_T_1")
    .size()
    .reset_index(name="total_batteries")
)

tract_counts["quintile"] = pd.qcut(
    tract_counts["total_batteries"],
    5,
    labels=["Q1 (Lowest)", "Q2", "Q3", "Q4", "Q5 (Highest)"]
```

```
)

Battery = Battery.merge(
    tract_counts[["CENSUS_T_1", "quintile"]],
    on="CENSUS_T_1",
    how="left"
)

Battery["month"] = Battery["Date"].dt.to_period("M").dt.to_timestamp()

quintile_time = (
    Battery
    .groupby(["month", "quintile"])
    .size()
    .reset_index(name="battery_count")
)

chart6 = (
    alt.Chart(quintile_time)
    .mark_line()
    .encode(
        x=alt.X("month:T", title="Month"),
        y=alt.Y("battery_count:Q", title="Number of Batteries"),
        color=alt.Color("quintile:N", title="Census Tract Quintile"),
    )
    .properties(
        width=700,
        height=400,
        title="Monthly Batteries by Census Tract Quintile"
    )
)

chart6
```



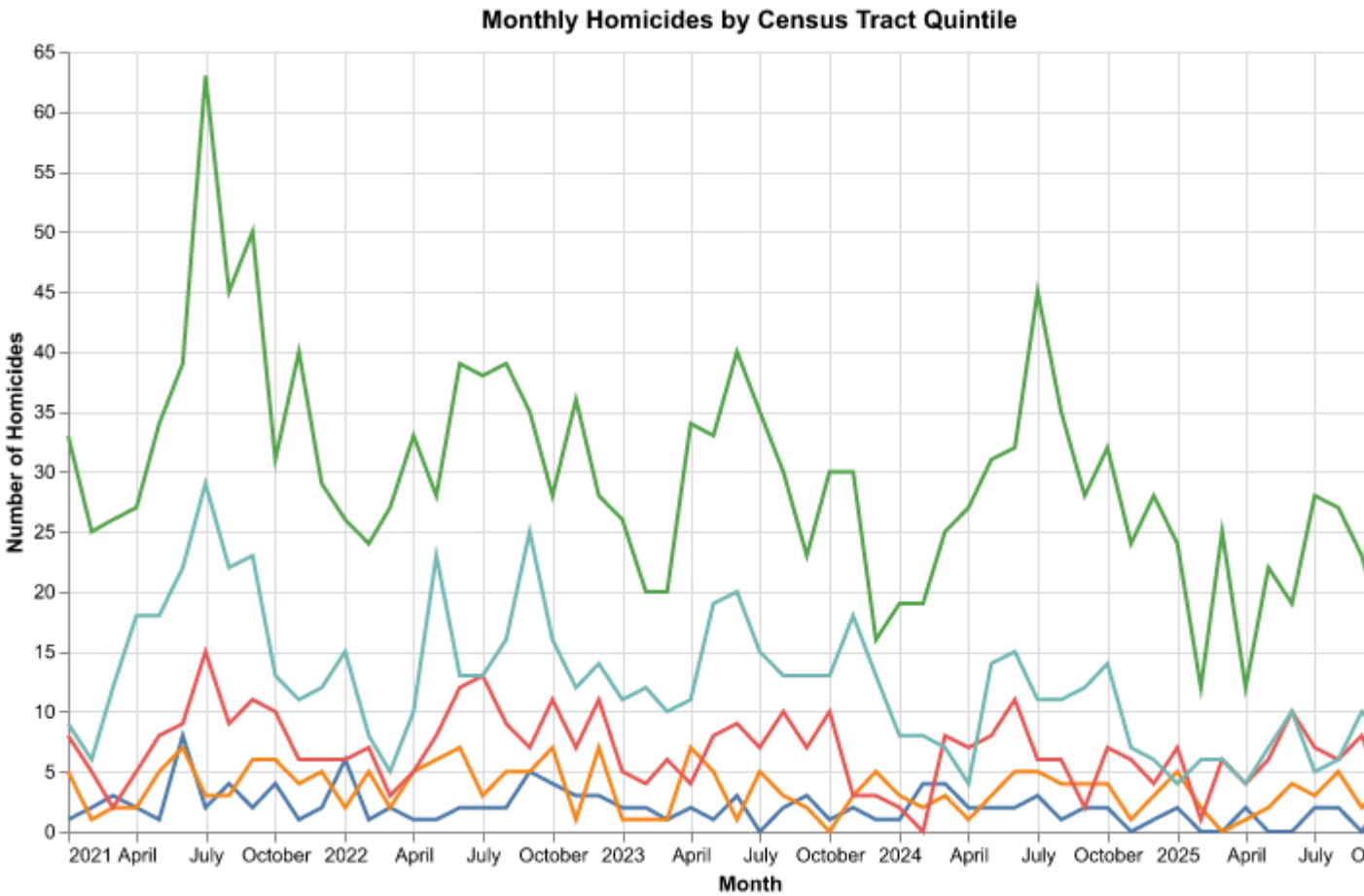

```
SA["Date"] = pd.to_datetime(SA["Date"])

if "quintile" in SA.columns:
    SA = SA.drop(columns=["quintile"])

tract_counts = (
    SA
    .groupby("CENSUS_T_1")
    .size()
    .reset_index(name="total_sexual_assaults")
)

tract_counts["quintile"] = pd.qcut(
    tract_counts["total_sexual_assaults"],
    5,
    labels=["Q1 (Lowest)", "Q2", "Q3", "Q4", "Q5 (Highest)"]
```

```
)

SA = SA.merge(
    tract_counts[["CENSUS_T_1", "quintile"]],
    on="CENSUS_T_1",
    how="left"
)

SA["month"] = SA["Date"].dt.to_period("M").dt.to_timestamp()

quintile_time = (
    SA
    .groupby(["month", "quintile"])
    .size()
    .reset_index(name="sexual_assault_count")
)

chart7 = (
    alt.Chart(quintile_time)
    .mark_line()
    .encode(
        x=alt.X("month:T", title="Month"),
        y=alt.Y("sexual_assault_count:Q", title="Number of Sexual Assaults"),
        color=alt.Color("quintile:N", title="Census Tract Quintile"),
    )
    .properties(
        width=700,
        height=400,
        title="Monthly Sexual Assaults by Census Tract Quintile"
    )
)

chart7
```




```
Homicide["Date"] = pd.to_datetime(Homicide["Date"])

if "quintile" in Homicide.columns:
    Homicide = Homicide.drop(columns=["quintile"])

tract_counts = (
    Homicide
    .groupby("CENSUS_T_1")
    .size()
    .reset_index(name="total_homicides")
)

tract_counts = tract_counts.sort_values("total_homicides")

tract_counts["quintile"] = pd.qcut(
    tract_counts["total_homicides"].rank(method="first"),
```

```
    5,
    labels=["Q1 (Lowest)", "Q2", "Q3", "Q4", "Q5 (Highest)"]
)

Homicide = Homicide.merge(
    tract_counts[["CENSUS_T_1", "quintile"]],
    on="CENSUS_T_1",
    how="left"
)

Homicide["month"] = Homicide["Date"].dt.to_period("M").dt.to_timestamp()

quintile_time = (
    Homicide
    .groupby(["month", "quintile"])
    .size()
    .reset_index(name="homicide_count")
)

chart8 = (
    alt.Chart(quintile_time)
    .mark_line()
    .encode(
        x=alt.X("month:T", title="Month"),
        y=alt.Y("homicide_count:Q", title="Number of Homicides"),
        color=alt.Color("quintile:N", title="Census Tract Quintile"),
    )
    .properties(
        width=700,
        height=400,
        title="Monthly Homicides by Census Tract Quintile"
    )
)

chart8
```




```
Crime["Primary Type"] = Crime["Primary Type"].replace(
    ["CRIM SEXUAL ASSAULT", "CRIMINAL SEXUAL ASSAULT"],
    "SEXUAL ASSAULT"
)
Crime["Date"] = pd.to_datetime(Crime["Date"])
Crime2023 = Crime[Crime["Date"].dt.year == 2023].copy()

PMH2023 = pd.read_csv("/Users/harryfranklin/Downloads/PMH2023.csv")
Crime2023["CENSUS_T_1"] = Crime2023["CENSUS_T_1"].astype(str)
PMH2023["GEOID"] = PMH2023["GEOID"].astype(str)
Crime2023 = Crime2023.merge(
    PMH2023[["GEOID", "PMH_2023"]],
    left_on="CENSUS_T_1",
    right_on="GEOID",
    how="left"
)
```

```
Crime2023 = Crime2023.drop(
    columns=[col for col in Crime2023.columns if "GEOID" in col],
    errors="ignore"
)

SES2023 = pd.read_csv("/Users/harryfranklin/Downloads/SES2023.csv")
Crime2023["CENSUS_T_1"] = Crime2023["CENSUS_T_1"].astype(str)
SES2023["GEOID"] = SES2023["GEOID"].astype(str)
Crime2023 = Crime2023.merge(
    SES2023[["GEOID", "SES_2023"]],
    left_on="CENSUS_T_1",
    right_on="GEOID",
    how="left"
)

Crime2023 = Crime2023.drop(
    columns=[col for col in Crime2023.columns if "GEOID" in col],
    errors="ignore"
)

HI2023 = pd.read_csv("/Users/harryfranklin/Downloads/HI2023.csv")
Crime2023["CENSUS_T_1"] = Crime2023["CENSUS_T_1"].astype(str)
HI2023["GEOID"] = HI2023["GEOID"].astype(str)
Crime2023 = Crime2023.merge(
    HI2023[["GEOID", "HDX_2019-2023"]],
    left_on="CENSUS_T_1",
    right_on="GEOID",
    how="left"
)

Crime2023 = Crime2023.drop(
    columns=[col for col in Crime2023.columns if "GEOID" in col],
    errors="ignore"
)

CHI2023 = pd.read_csv("/Users/harryfranklin/Downloads/CHI2023.csv")
Crime2023["CENSUS_T_1"] = Crime2023["CENSUS_T_1"].astype(str)
CHI2023["GEOID"] = CHI2023["GEOID"].astype(str)
Crime2023 = Crime2023.merge(
    CHI2023[["GEOID", "CHACFZN_2023"]],
    left_on="CENSUS_T_1",
    right_on="GEOID",
```

```
    how="left"
)

Crime2023 = Crime2023.drop(
    columns=[col for col in Crime2023.columns if "GEOID" in col],
    errors="ignore"
)

Crime2023 = Crime2023.rename(columns={"HDX_2019-2023": "HDX"})
Crime2023 = Crime2023.rename(columns={"CHACFZN_2023": "COI"})

Crime2023 = Crime2023.rename(columns={"PMH_2023": "PMH"})
Crime2023 = Crime2023.rename(columns={"SES_2023": "SES"})
Crime2023 = Crime2023.dropna(subset=["Population"])

Crime2023["Population"] = (
    Crime2023["Population"]
    .astype(str)            # ensure it's a string first
    .str.replace(",", "", regex=False)
    .str.strip()
)
Crime2023["Population"] = pd.to_numeric(Crime2023["Population"], errors="coerce")

tract_vars = (
    Crime2023.groupby("CENSUS_T_1", as_index=False)
    .agg({
        "PMH": "first",
        "SES": "first",
        "HDX": "first",
        "COI": "first",
        "Population": "first"
    })
)

cols_to_numeric = ["PMH", "SES", "HDX", "COI"]
for col in cols_to_numeric:
    Crime2023[col] = pd.to_numeric(Crime2023[col], errors="coerce")

# Clean Population separately
Crime2023["Population"] = (
    Crime2023["Population"].astype(str)
    .str.replace(",", "", regex=False)
```

```python
    .str.strip()
)
Crime2023["Population"] = pd.to_numeric(Crime2023["Population"], errors="coerce")

Crime2023["Date"] = pd.to_datetime(Crime2023["Date"])

# Count crimes per tract
tract_crime = (
    Crime2023
    .groupby("CENSUS_T_1")
    .size()
    .reset_index(name="crime_count")
)

# Aggregate tract-level variables
tract_vars = (
    Crime2023.groupby("CENSUS_T_1", as_index=False)
    .agg({
        "PMH": "first",
        "SES": "first",
        "HDX": "first",
        "COI": "first",
        "Population": "first"
    })
)

# Merge counts with tract variables
tract_data = tract_crime.merge(tract_vars, on="CENSUS_T_1", how="left")

# Keep only tracts with positive population
tract_data = tract_data[tract_data["Population"] > 0]

# Compute crime rate
tract_data["crime_rate"] = (tract_data["crime_count"] / tract_data["Population"]) * 1000

# Compute correlations
corr_cols = ["crime_rate", "PMH", "SES", "HDX", "COI"]
correlations = tract_data[corr_cols].corr(method="pearson")
print(correlations["crime_rate"].sort_values(ascending=False))
```

```
crime_rate    1.000000
PMH           0.715803
```

```
SES           0.703464
HDX           0.547125
COI          -0.599104
Name: crime_rate, dtype: float64
```

```python
import numpy as np
from scipy.stats import pearsonr

results = []

health_vars = ["PMH", "SES", "HDX", "COI"]

for var in health_vars:
    df = tract_data[["crime_rate", var]].dropna()
    n = len(df)

    r, p = pearsonr(df["crime_rate"], df[var])

    # Standard error formula
    se = np.sqrt((1 - r**2) / (n - 2))

    results.append({
        "Variable": var,
        "Correlation (r)": r,
        "Std Error": se,
        "P-value": p,
        "N": n
    })

corr_df = pd.DataFrame(results)
print(corr_df)

Banana = corr_df
Banana = Banana.reset_index()
Banana.columns = ["Number", "Health Indicator", "Correlation Coefficient", "Std Error", "P-Va
Banana = Banana[Banana["Health Indicator"] != "crime_rate"]
Banana["Health Indicator"] = Banana["Health Indicator"].replace({
    "PMH": "% of adults who reported >13 days of poor mental health in the past month",
    "SES": "Percent of adults who report rarely or never getting the social and emotional sup
    "HDX": "Hardship Index",
    "COI": "Childhood Opportunity Index"
})
```

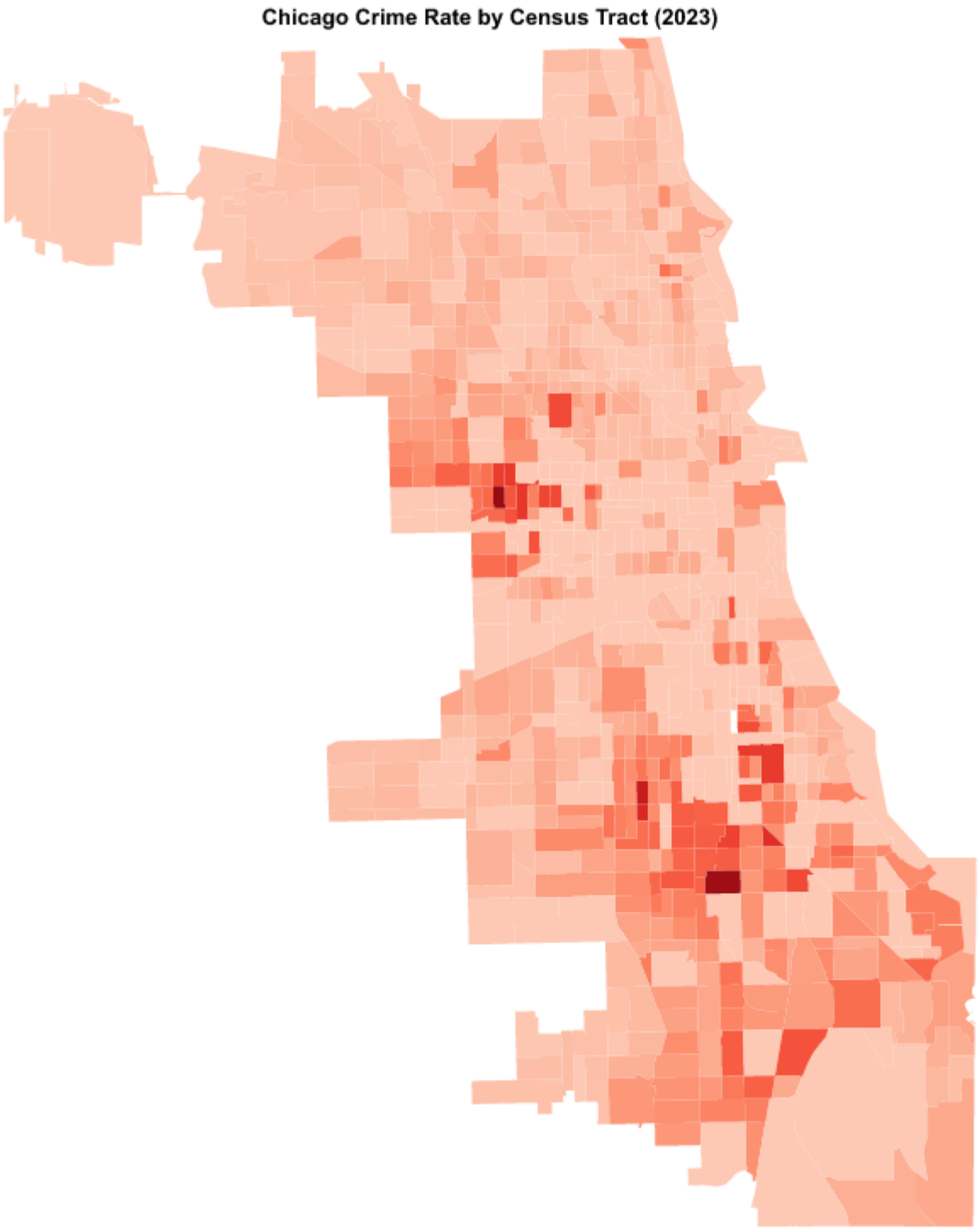
```
Banana
Banana = Banana.drop(columns=["Number"])
Banana
```

```
  Variable  Correlation (r)  Std Error       P-value    N
0      PMH         0.715803   0.029830  1.631682e-87  550
1      SES         0.703464   0.030361  2.622715e-83  550
2      HDX         0.547125   0.035757  2.887265e-44  550
3      COI        -0.599104   0.034203  7.019359e-55  550
```

	Health Indicator	Correlati
0	% of adults who reported >13 days of poor mental health in the past month	0.715803
1	Percent of adults who report rarely or never getting the social and emotional support they need	0.703464
2	Hardship Index	0.547125
3	Childhood Opportunity Index	-0.59910

```
import geopandas as gpd

PATH = "data/Census_20Tracts/Census_Tracts.shp"
gdf = gpd.read_file(PATH)
gdf = gdf.to_crs(epsg=4326)

import altair as alt
import pandas as pd
import json

Crime2023["CENSUS_T_1"] = Crime2023["CENSUS_T_1"].astype(str)
gdf["CENSUS_T_1"] = gdf["CENSUS_T_1"].astype(str)


tract_crimes = (
    Crime2023.groupby("CENSUS_T_1")
    .size()
    .reset_index(name="total_crimes")
)

tract_crimes = tract_crimes.merge(
    Crime2023[["CENSUS_T_1", "Population"]].drop_duplicates(),
    on="CENSUS_T_1",
```

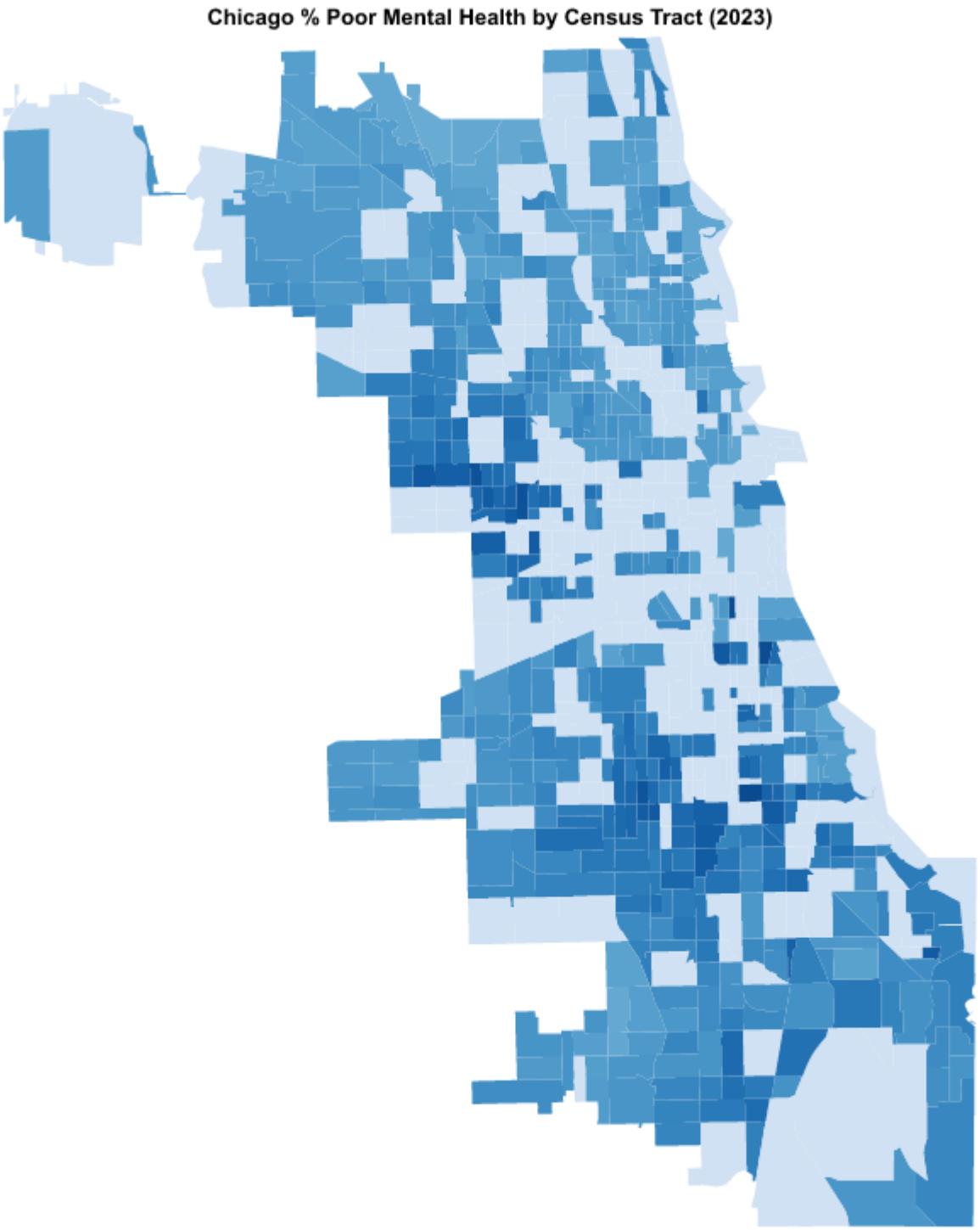
```
    how="left"
)

tract_crimes["crime_rate"] = (tract_crimes["total_crimes"] / tract_crimes["Population"]) * 10

gdf = gdf.drop(columns=[col for col in gdf.columns if "crime_rate" in col], errors="ignore")
gdf = gdf.merge(
    tract_crimes[["CENSUS_T_1", "crime_rate"]],
    on="CENSUS_T_1",
    how="left"
)

gdf["crime_rate"] = gdf["crime_rate"].fillna(0)

gdf_json = json.loads(gdf.to_json())

chart9 = alt.Chart(alt.Data(values=gdf_json['features'])).mark_geoshape().encode(
    color=alt.Color(
        'properties.crime_rate:Q',
        scale=alt.Scale(scheme='reds'),
        title='Crime Rate per 1,000 Residents'
    ),
    tooltip=[
        alt.Tooltip('properties.CENSUS_T_1:N', title='Census Tract'),
        alt.Tooltip('properties.crime_rate:Q', title='Crime Rate')
    ]
).properties(
    width=700,
    height=700,
    title='Chicago Crime Rate by Census Tract (2023)'
).project('mercator')  # ← use mercator projection to orient north-up

chart9
```


Chicago Crime Rate by Census Tract (2023)

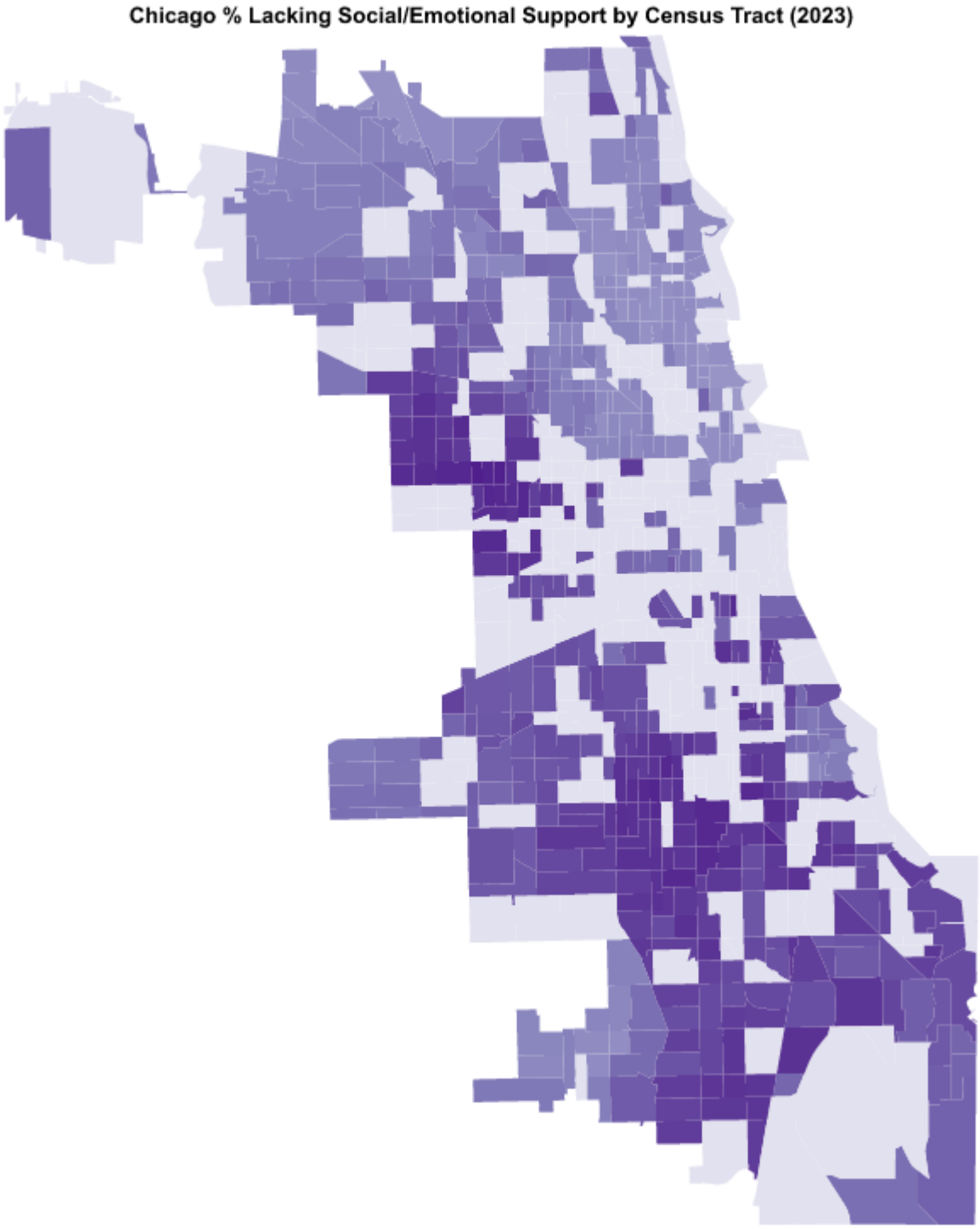
```python
Crime2023["CENSUS_T_1"] = Crime2023["CENSUS_T_1"].astype(str)
gdf["CENSUS_T_1"] = gdf["CENSUS_T_1"].astype(str)

# Compute crime rate per 1,000 residents
tract_data = (
    Crime2023.groupby("CENSUS_T_1")
    .agg(
        total_crimes=('CENSUS_T_1', 'size'),
        Population=('Population', 'first'),
        PMH=('PMH', 'first'),
        SES=('SES', 'first'),
        HDX=('HDX', 'first'),
        COI=('COI', 'first')
    )
    .reset_index()
)

tract_data["crime_rate"] = (tract_data["total_crimes"] / tract_data["Population"]) * 1000

gdf = gdf.merge(
    tract_data[['CENSUS_T_1', 'crime_rate', 'PMH', 'SES', 'HDX', 'COI']],
    on='CENSUS_T_1',
    how='left'
)

for col in ['PMH','SES','HDX','COI']:
    gdf[col] = gdf[col].fillna(0)

gdf_json = json.loads(gdf.to_json())
```

```python
def plot_chicago_heatmap(var, title=None, color_scheme='reds'):
    if title is None:
        title = f'Chicago {var} by Census Tract (2023)'

    chart = alt.Chart(alt.Data(values=gdf_json['features'])).mark_geoshape().encode(
        color=alt.Color(
            f'properties.{var}:Q',
            scale=alt.Scale(scheme=color_scheme),
            title=var.replace('_',' ')
        ),
        tooltip=[
            alt.Tooltip('properties.CENSUS_T_1:N', title='Census Tract'),
```

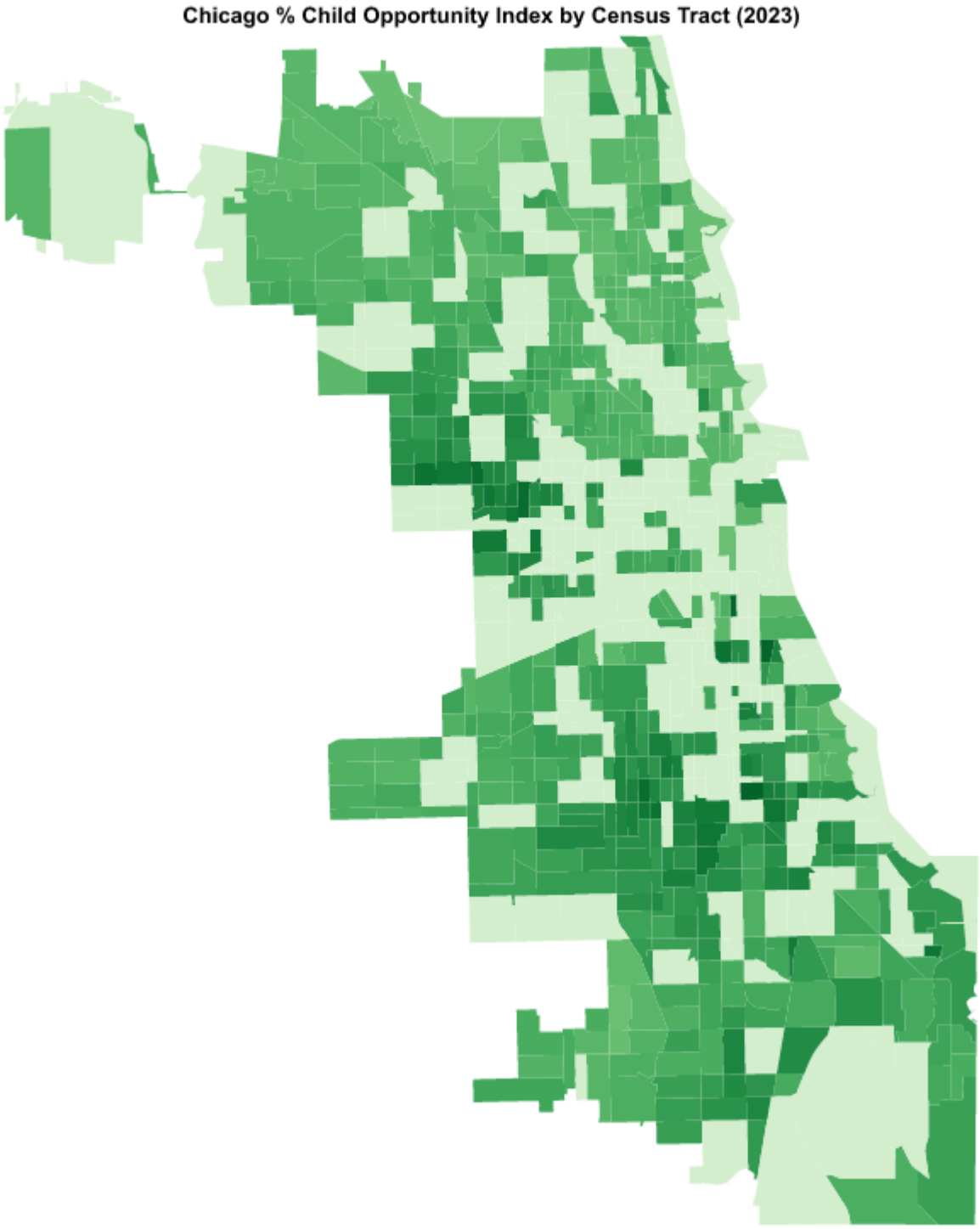
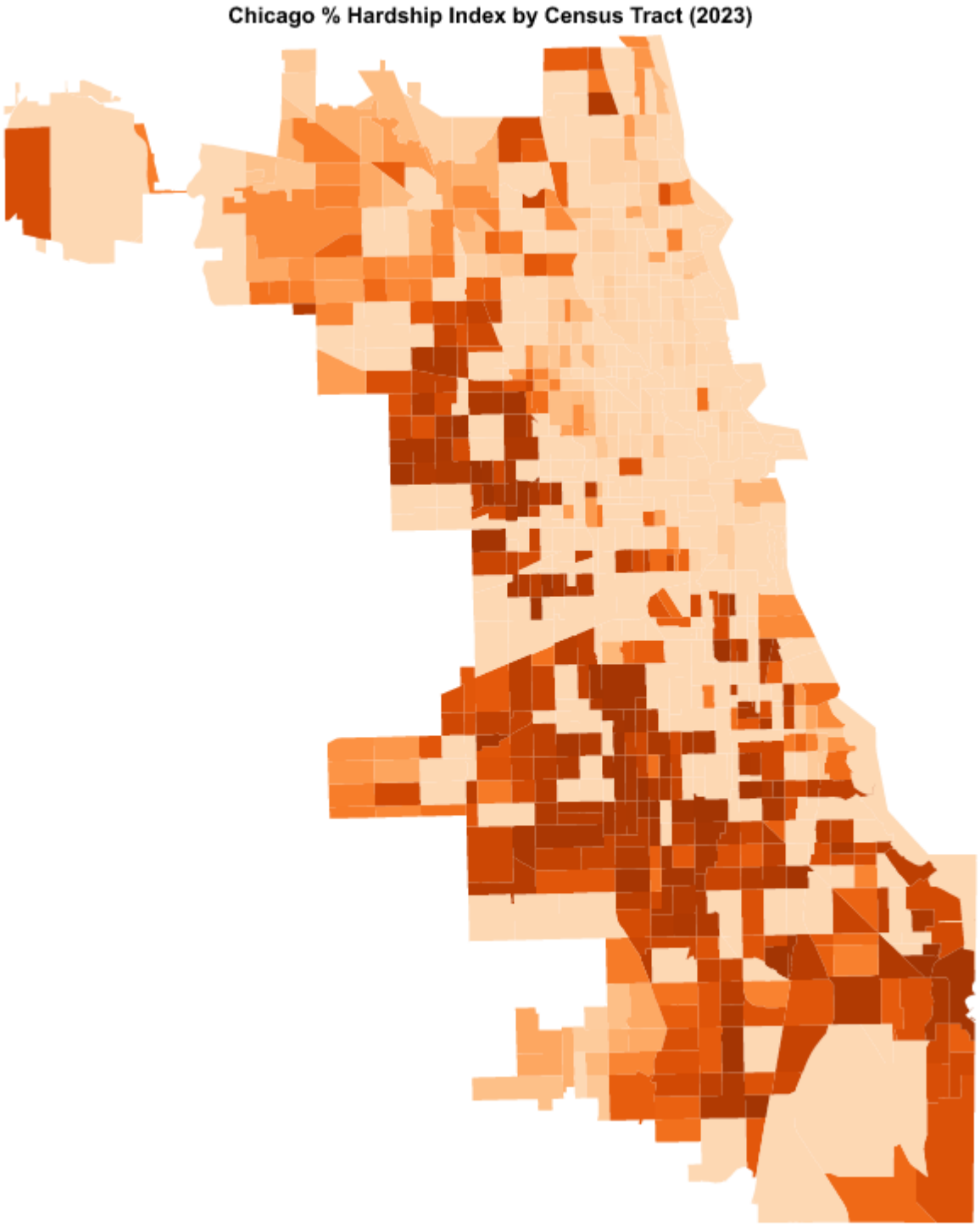
```
            alt.Tooltip(f'properties.{var}:Q', title=var.replace('_',' '))
        ]
    ).properties(
        width=700,
        height=700,
        title=title
    ).project('mercator')

    return chart

plot_chicago_heatmap('PMH', 'Chicago % Poor Mental Health by Census Tract (2023)', color_sche
```


Chicago % Poor Mental Health by Census Tract (2023)

```
plot_chicago_heatmap('SES', 'Chicago % Lacking Social/Emotional Support by Census Tract (2023
```



```
plot_chicago_heatmap('PMH', 'Chicago % Child Opportunity Index by Census Tract (2023)', colo
```



```
plot_chicago_heatmap('HDX', 'Chicago % Hardship Index by Census Tract (2023)', color_scheme=
```